Máster en Data Science y Big Data

UNIVERSIDAD D SEVILLA

Trabajo de Fin de Máster

Predicción del precio de Airbnb en Nueva York

Un precio de partida para cada anuncio nuevo

Hugo Montilla

Octubre de 2026



CFP Centro de Formación Permanente

Índice

Documentación para analistas 3
1. Objetivos del proyecto 3
2. Principales logros obtenidos 5
3. Aproximación elegida 6
4. Descripción del modelo 8
5. Resultados principales soportados por datos 8
6. Detalles del modelo 13
7. Recomendaciones 16

Documentación para el sponsor 18
Resumen ejecutivo 18
1. Objetivos del proyecto 18
2. Principales logros obtenidos 19
3. Aproximación elegida 19
4. Descripción del modelo 19
5. Resultados principales soportados por datos 20
6. Detalles del modelo 23
7. Recomendaciones 23

Apéndices 25
A. Mapa del repositorio 25
B. Notebooks (orden de ejecución) 25
C. Módulos de scripts/ 25
D. Reproducción 26
E. Uso del modelo guardado 26
F. Documentación de trazabilidad 27
G. Tablas complementarias 27

Documentación para analistas

Predicción del precio por noche de un anuncio nuevo de Airbnb en Nueva York

> Las cifras proceden de `outputs/`, `models/` y de las conclusiones de los notebooks; el repositorio completo acompaña a esta memoria como apéndice y permite reproducir cada resultado. Salvo que se indique otra cosa: **CV** = validación cruzada de 5 folds sobre el train, agrupada por anfitrión; **test** = 7.384 anuncios apartados antes de cualquier análisis del precio y evaluados una sola vez; **RMSE** = raíz del error cuadrático medio sobre `log(price)`.

1. Objetivos del proyecto

Situación

Airbnb publica para cada anuncio su precio por noche junto con las características del alojamiento, su ubicación y el perfil del anfitrión. El conjunto de datos aportado (`listings.csv`, 279.712 anuncios de 10 ciudades, y `reviews.csv`, con datos hasta 2021) recoge esa información, pero no dice cuánto *debería* cobrar un anuncio nuevo. Un anfitrión que publica por primera vez fija el precio mirando anuncios parecidos, sin una referencia objetiva.

Complicación

El precio de un alojamiento depende de muchas cosas a la vez y de forma poco intuitiva. Durante el análisis exploratorio se comprobó repetidas veces que **la lectura marginal engaña**: la densidad de huéspedes por dormitorio, el volumen de reseñas o la estancia mínima cambiaban de signo al comparar dentro del mismo tipo de producto. Una regla sencilla ("un piso entero para cuatro cuesta X") deja errores grandes, y combinar a mano decenas de atributos no es viable. Es un problema natural para un modelo supervisado que aprenda esas relaciones a partir de los datos.

Objetivos

1. **Predecir el precio por noche de un anuncio nuevo** en Nueva York con la información disponible en el momento de publicarlo (alojamiento, ubicación y perfil del anfitrión), sin usar nada que solo exista después (reseñas).
2. **Acompañar la predicción de un rango** que contenga el precio real en el 80% de los casos.
3. **Explicar qué determina el precio** y dónde es fiable el modelo y dónde no.

Cambio de objetivos durante el proyecto: de 10 ciudades a Nueva York

El planteamiento inicial era un estudio del mercado internacional. El análisis de `0-INTRODUCCION` lo descartó por dos razones, ambas medidas en los datos:

- **No hay un objetivo común.** `price` viene en la moneda local de cada ciudad y el fichero no lo documenta: la mediana va de 65 (Roma) a 1.100 (Bangkok), ×17. Al aplicar un tipo de cambio aproximado, la dispersión baja a ×3, lo que confirma que la diferencia era la divisa. Convertir exigiría el tipo de cambio de la fecha del volcado, que no está, y no resolvería las diferencias de poder adquisitivo.

- **El precio no se comporta igual en todas las ciudades**, incluso con medidas que la moneda no afecta (Figura 1): doblar la capacidad multiplica el precio por 1,5 en Bangkok y por 2,2 en Ciudad del Cabo, y una habitación privada cuesta el 84% de un piso entero en Bangkok frente al 39% en Sídney. Un modelo único necesitaría interacciones de la ciudad con casi todo.

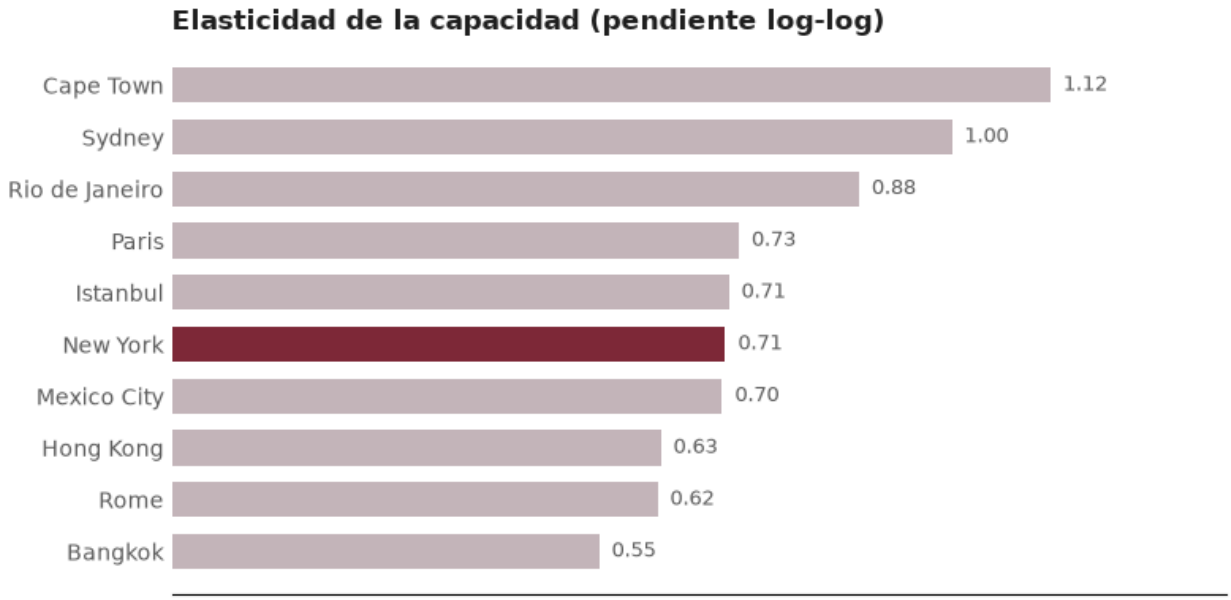


Figura 1. La capacidad no vale lo mismo en cada ciudad: la pendiente log-log entre capacidad y precio se duplica de Bangkok a Ciudad del Cabo. Fuente: 0-INTRODUCCION, §3.

Se eligió **Nueva York** porque es la única ciudad con `district` completo (en las otras nueve la columna está vacía), la que sitúa cada anuncio con más detalle (220 barrios), tiene muestra de sobra (37.012 anuncios, la segunda tras París) y su precio está en una sola divisa.

Formulación técnica

Regresión supervisada sobre **`log(price)`**. El logaritmo reduce la asimetría del precio de 21,1 a 0,7 (Figura 2) y convierte el error en relativo, que es lo que importa al recomendar un precio. Al deshacer la transformación, `exp(ŷ)` estima la **mediana** del precio de un anuncio con esas características, no su media. La métrica principal de comparación es el RMSE en log; para el negocio se reporta además el error absoluto en dólares (MAE y mediana del error).

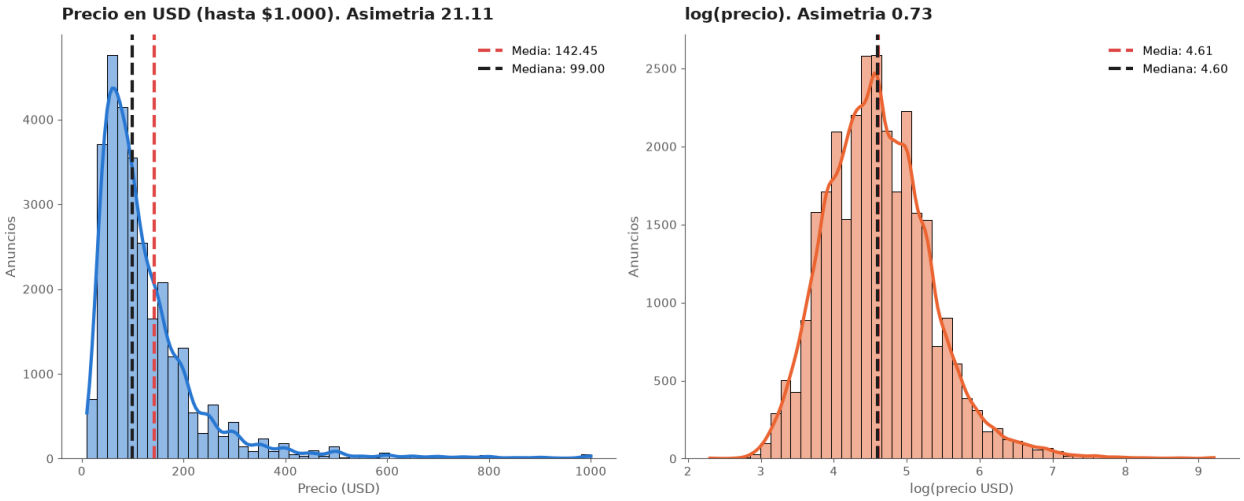


Figura 2. El precio tiene una cola derecha muy larga; en logaritmo la distribución es casi simétrica. Fuente: 2-ANALISIS, §2.1.

Se definieron dos escenarios de uso: **B (principal)**, un anuncio nuevo de cualquier anfitrión, con su perfil (antigüedad, tasas de respuesta, cartera…); y **A (contraste)**, un anfitrión nuevo sin historial, que prescinde de ese bloque. La diferencia entre ambos mide cuánto vale conocer al anfitrión.

2. Principales logros obtenidos

- **Un modelo que predice el precio de un anuncio nuevo con un error típico de 21$.** El modelo de referencia, CatBoost, obtiene en el test un RMSE en log de **0,429** (IC 95% 0,408-0,451) y una mediana del error de **21,1$** (Tabla 2). Todas las métricas quedan dentro de ±2 desviaciones típicas de la CV: no hay señal de sobreajuste a la validación.
- **Una mejora clara sobre las referencias razonables.** En CV, el modelo reduce el RMSE un **22%** frente a la mejor regla ingenua y la mediana del error de 29$ a 21,5$.
- **Un rango de precio calibrado.** El intervalo 10%-90%, calibrado con regresión cuantílica conformal (CQR), cubre el **77,9%** de los precios del test (IC 95% 72,3-81,5), con el 80% nominal dentro del intervalo de confianza.
- **Un diagnóstico honesto de dónde falla.** El error se concentra en los anuncios caros: en el 20% más caro la cobertura del rango baja al 55,9% y los 20 peores errores son pisos de lujo infravalorados.
- **Una lectura clara de qué determina el precio:** el producto (tipo, capacidad, dormitorios) domina, seguido de la ubicación; el perfil del anfitrión y las amenities aportan algo parecido entre sí, y las condiciones de reserva casi nada.
- **Un proceso reproducible y sin fuga de información:** partición fija agrupada por anfitrión y un test abierto una sola vez tras comprobar su huella MD5.

3. Aproximación elegida

3.1 El problema y la idea que ordena todo el proceso

El problema es de **regresión supervisada** sobre `log(price)` con datos tabulares mixtos (numéricos, categóricos de alta cardinalidad y 56 binarias de amenities). La decisión de diseño que condiciona el resto es una **barrera física contra la fuga de información**: el test se separa antes de mirar el precio, los notebooks de análisis y modelado solo leen el train, y el test solo lo abre el notebook del modelo final.

```
listings.csv ─► 0-INTRODUCCION (alcance: Nueva York)
             ─► 1-EXPLORACION (calidad de datos sin mirar el precio)
             ─► 2-ANALISIS (EDA del precio, solo train)
             ─► 3-MODELADO (registro de experimentos)
             ─► 4-MODELO_FINAL (dos finalistas)
```

3.2 Extracción, transformación y carga

- **Extracción:** filtrado de Nueva York (37.012 anuncios y 847.727 reseñas, enlazadas por `listing_id`).
- **Calidad (sin mirar el precio):** sin duplicados de extracción; 11 anuncios idénticos salvo el identificador (alojamientos distintos de un mismo edificio, se conservan); 13 anuncios con capacidad 0 y 15 con precio nulo o 0, eliminados (un precio nulo no es una observacións).
- **Precios de bloqueo:** se excluyen precios ≥ 9.999$ y habitaciones a más de 1.000$ la noche (48 anuncios del train y 14 del test), con umbrales fijados en el train. Se descartó recortar por el percentil 99 (850$) porque eliminaba el 18,8% de los pisos para 9 o más huéspedes, es decir, el lujo legítimo que el modelo debe aprender. Resultado: **36.922 anuncios válidos, 29.538 en train y 7.384 en test**.
- **Transformación:** toda la preparación está en `PreparadorListings` (`scripts/preparacion_datos.py`), con `fit`/`transform` ajustado **solo con el train** y congelado en `data/modelado/preparador.json`. Aprende la ventana de amenities, los tipos de propiedad frecuentes y las medianas para imputar dormitorios.
- **Variables construidas:** `es_estudio` (el 70% de los pisos enteros sin dato de dormitorios lleva *studio* en el título, y cuestan un 12% menos que un piso de un dormitorio de igual capacidad, por eso al imputar `bedrooms` a 1 añadimos esta variable); `host_ambito` (dónde vive el anfitrión, normalizado desde texto libre); `dist_centro_km` `n_anuncios_ny` (tamaño de la cartera del anfitrion); `reputacion_cartera` (puntuación media de los *otros* anuncios del anfitrión, disponible al publicar); y 56 amenities binarias con prevalencia entre el 3,5% y el 97% (criterio ciego al precio), las que aparecen en mas del 97% y menos del 3.5% no aportan informacion al estar presentes (o no presentes) en la mayoria de anuncios.

3.3 Partición y validación

- **Agrupada por anfitrión.** El 38,1% de los anuncios pertenece a anfitriones con varios pisos y el 23,7% tiene un "gemelo" en su cartera (mismo tipo, capacidad, dormitorios y barrio). Partir por filas dejaría gemelos a ambos lados y el error saldría optimista; es una fuga reconocida en la literatura sobre datos jerárquicos (Roberts et al., 2017; Kapoor y Narayanan, 2023).
- **Estratificada por `room_type` × `district`.** En 50 particiones simuladas, sin estratificar el test llegaba a tener el doble de habitaciones de hotel de las esperadas; con el estrato combinado la desviación queda por debajo de 0,1 puntos en las 18 celdas.

- **Test del 20%, semilla 42, guardado con huella MD5** en `data/particion/`. El script se niega a sobrescribirlo.
- **CV de 5 folds × 3 repeticiones** con el mismo agrupamiento y estrato, guardada en `outputs/folds_cv.csv`. Las 15 estimaciones se usan para seleccionar variables; la primera repetición (5 folds), para comparar modelos y ajustar hiperparámetros. Al compartir folds, todas las comparaciones son **pareadas**.

3.4 Protocolo de experimentación

- **Escalera de complejidad:** referencias ingenuas → lineal → kNN y árbol → Random Forest y HGB → SVR y CatBoost. Cada peldaño responde a una pregunta (cuánto aporta el producto, la ubicación, la no linealidad).
- **Reglas de decisión fijadas antes de mirar los resultados**, recogidas en la bitácora con fecha: la regla de selección de variables, la de elección del modelo final (menor RMSE medio de CV en el escenario B) y el margen de equivalencia práctica (±0,005 de RMSE en log).
- **Registro de todos los experimentos**, también los que salieron mal: más de 500 experimentos en `outputs/experimentos.csv`, una fila por fold, con variables, hiperparámetros y tiempos.
- **Todo lo que aprende de los datos va dentro del `Pipeline`** de cada fold (codificaciones, imputación, escalado).

3.5 Restricciones encontradas

- **Sin información de calidad o lujo** más allá de las amenities: no hay superficie, fotos ni descripción.
- **Normativa local:** la *Multiple Dwelling Law* de Nueva York explica que el 64% de los anuncios exija 30 noches mínimas; no altera el precio por noche, pero condiciona el tipo de mercado.
- **Cómputo:** la búsqueda de hiperparámetros de CatBoost se recortó de 40 a 10 candidatos para que el plan cupiera en el calendario (se declara que se decidió viendo resultados parciales; el óptimo ya era plano).
- **Sin datasets externos:** no se han usado otros datos para enriquecer los del cliente. `reviews.csv` solo se usó para fechar la instantánea y caracterizar la actividad, no como variable.

3.6 Tecnologías

- **Lenguaje:** Python 3.13.
- **Datos:** pandas 3.0 y pyarrow.
- **Modelado:** scikit-learn 1.9 (pipelines, `StratifiedGroupKFold`, HGB, Random Forest, kNN, SVR con Nystroem, `TargetEncoder`) y CatBoost 1.2 (incluido SHAP nativo).
- **Estadística:** statsmodels (OLS con errores agrupados).
- **Datos mixtos:** prince (FAMD) y una implementación propia de PCAmix.
- **Visualización:** matplotlib y seaborn.
- **Cómputo:** Google Colab con GPU, para repetir la búsqueda de CatBoost como prueba de velocidad.

4. Descripción del modelo

Metodología	Predecir la mediana del precio por noche de cada anuncio a partir de sus características al publicarse; aprender la relación con anuncios existentes y medirla en anuncios de anfitriones que el modelo no ha visto
Modelo	*Gradient boosting* con árboles simétricos (CatBoost), con un intervalo del 80% formado por dos modelos cuantílicos calibrados con CQR. Alternativa equivalente: HGB de scikit-learn
Variable dependiente	`log(price)`, precio por noche en USD
Variables explicativas	79 que diferenciamos en cinco bloques: producto, ubicación, condiciones de reserva, perfil del anfitrión y amenities
Frente a la referencia	Mejora un 22% en RMSE a la mejor regla ingenua (mediana por tipo × capacidad) y un 4% al mejor modelo lineal
Conocer al anfitrión	Sin su perfil (escenario A), el RMSE sube un 3,7% (HGB) y un 4,0% (lineal)

5. Resultados principales soportados por datos

5.1 Lo que dicen los datos antes de modelar (EDA sobre el train)

El análisis exploratorio se hizo **solo con el train** y con estadísticos robustos (medianas, Kruskal-Wallis, Mann-Whitney con corrección de Holm, Spearman), ocultando celdas con menos de 30 anuncios.

- **El tipo de alojamiento es lo que más separa el precio** ($\varepsilon^2 = 0{,}41$ en Kruskal-Wallis): pisos enteros 140$ de mediana, habitaciones privadas 60$.
- **La ubicación opera en tres niveles:** distrito ($\varepsilon^2 = 0{,}09$; Manhattan 120$ frente a Bronx 67$), barrio ($\eta^2 = 0{,}19$) y distancia al centro, con un gradiente que persiste dentro de cada distrito (Figura 3).
- **Hay economía de escala:** el precio por plaza cae un 50% de 1 a 7 o más huéspedes, y un huésped adicional vale 36$ en Manhattan frente a 20$ en el Bronx.
- **La lectura marginal engaña** (Figura 4): la densidad de huéspedes por dormitorio correlaciona positivamente con el precio ($\rho = +0{,}38$), pero a igual capacidad más dormitorios es más precio en todos los tramos.
- **Señales menores:** la reserva inmediata se asocia a un precio un 7% más bajo a igual producto y zona; los anuncios sin tiempo de respuesta (inactivos, la mitad del train) piden un 18% más; los superhosts no cobran más, suponemos que tienen una mayor rotacion.

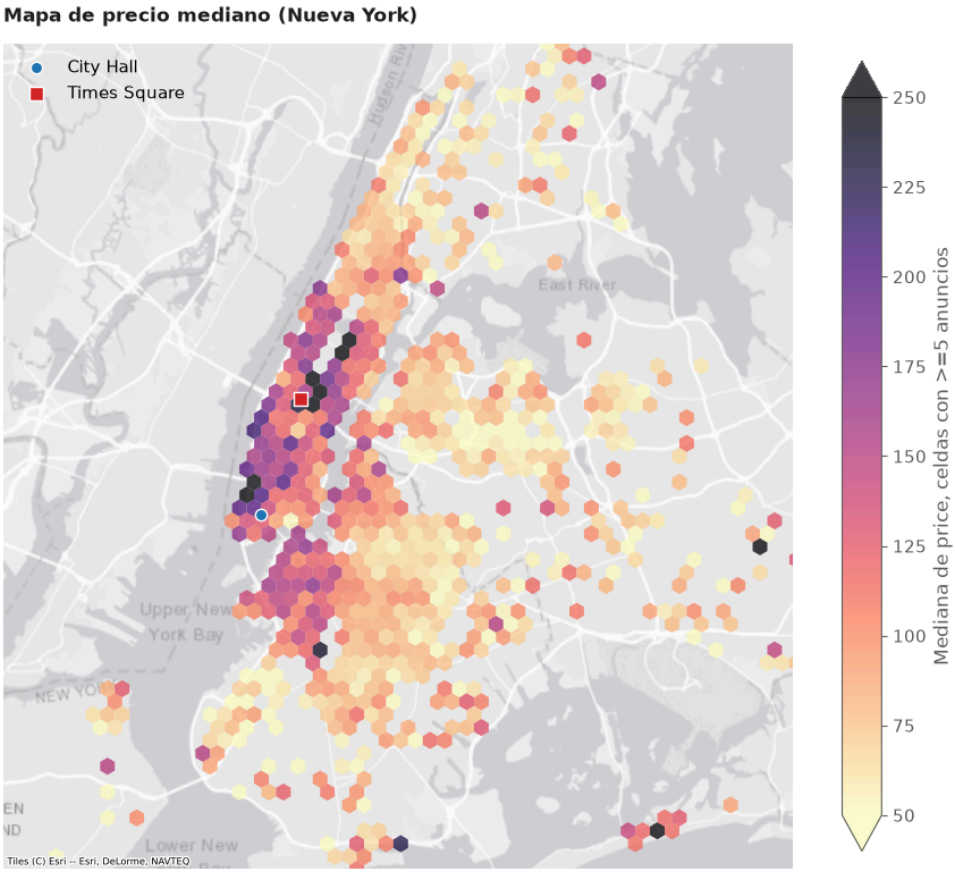


Figura 3. El precio se concentra en el sur y centro de Manhattan y cae hacia la periferia. Fuente: 2-ANALISIS, §7.

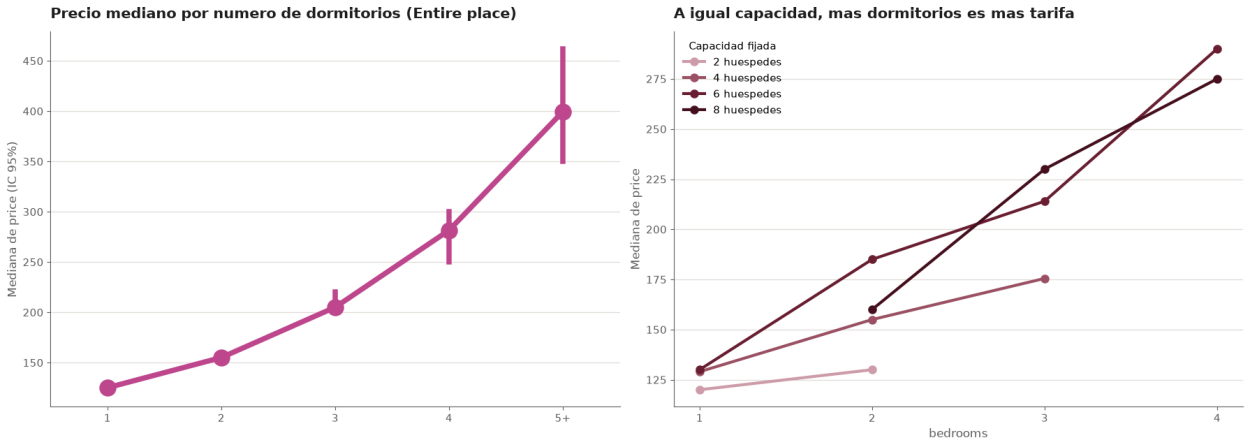


Figura 4. A igual capacidad, cada dormitorio adicional sube el precio: la correlación marginal positiva de la densidad era un efecto de la capacidad. Fuente: 2-ANALISIS, §4.2.

5.2 La escalera de modelos

Referencias ingenuas. Predecir siempre la mediana desde el global por agrupaciones. Captura casi todo el comportamiento aditivo.

Modelo lineal. La escalera aditiva (Figura 5) muestra cuánto aporta cada bloque: producto completo 0,514 →+ ubicación 0,475 (el mayor salto) → + condiciones 0,475 (nada) → +

anfitrión 0,451 → + amenities 0,437. Las interacciones apenas aportan y los *splines* sin penalizar hacen la OLS inestable (rango incompleto; predicciones absurdas fuera del rango del train). La regularización no mejora la precisión (Ridge, Lasso y ElasticNet empatan con n ≫ p), pero estabiliza el diseño. **El lineal elegido (Lasso, 79 variables) llega a 0,434**: un modelo aditivo ya explica el 63% de la varianza del log-precio.

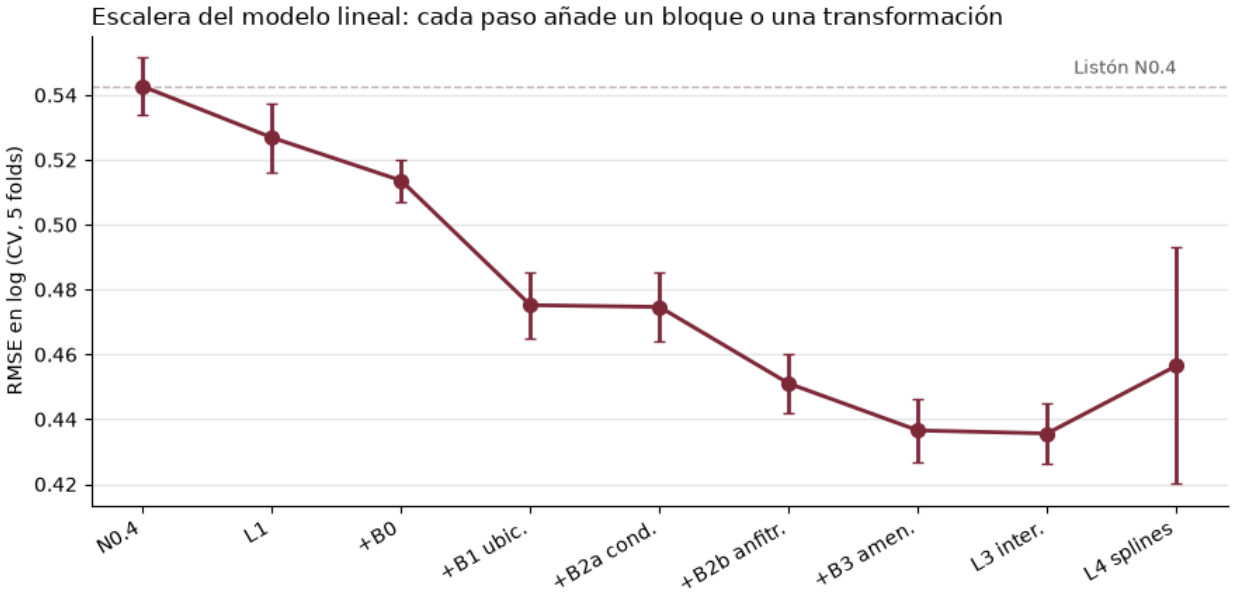


Figura 5. La ubicación y el perfil del anfitrión son los dos grandes saltos del modelo aditivo después del producto; las condiciones de reserva no aportan. Fuente: 3-MODELADO, §5.

Coeficientes hedónicos. Una OLS interpretable, con errores estándar agrupados por anfitrión, permite leer asociaciones condicionadas en porcentaje del precio: una habitación privada cuesta un 40% menos que un piso entero comparable; cada dormitorio suma un 16% y cada huésped un 9%. Agrupar los errores por anfitrión los hace 1,5 veces mayores y evita declarar significativossiete efectos que no lo son. Entre las amenities, las de edificio o vivienda de gama alta (gimnasio,chimenea) se asocian a primas de alrededor del 20%, y las de cocina básica y estancia larga, a precios algo menores.

Los diagnósticos de la OLS (Figura 7) muestran heterocedasticidad, una cola derecha pesada (anuncios mucho más caros de lo predicho: el lujo que las variables no ven) y manchas espaciales en los residuos. Es el hueco que deben cubrir los modelos no lineales con las coordenadas.

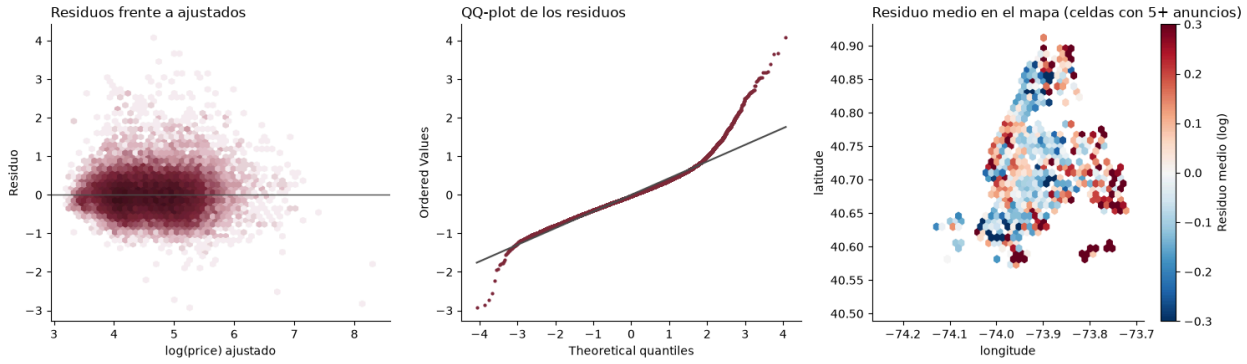


Figura 7. La OLS deja una cola de anuncios caros sin explicar y estructura espacial en los residuos. Fuente: 3-MODELADO, §5.3.

Modelos no lineales y ensembles. La Tabla 1 resume la comparación con los mismos 5 folds.

Tabla 1. Comparación de modelos en CV (5 folds, escenario B). Fuente: `outputs/experimentos.csv`.

Modelo	Variables	RMSE log (media ± sd)	R^2 log	Mediana error ($)
CatBoost ajustado	79	**0,416 ± 0,009**	0,657	21,5
HGB ajustado	79	0,419 ± 0,010	0,653	21,5
Random Forest ajustado	79	0,428 ± 0,012	0,638	22,0
SVR (kernel Nystroem)	79	0,432 ± 0,012	0,631	22,7
Lineal (Lasso)	79	0,434 ± 0,009	0,627	22,6
Árbol de decisión (prepoda)	79	0,472 ± 0,014	0,560	23,8
kNN, 5 variables (K1)	5	0,474 ± 0,012	0,556	24,8
kNN con PCAmix (K3)	65	0,481 ± 0,012	0,543	24,9
Mediana por tipo × capacidad	2	0,535 ± 0,009	0,434	29,0
Mediana por barrio × tipo	3	0,543 ± 0,009	0,418	26,0
Mediana global	0	0,711 ± 0,009	0,000	43,8

La lectura de la escalera es que **la mayor parte de la señal ya la recoge el modelo aditivo**: el lineal queda a un 4,3% del mejor. Lo que añaden los *boosting* son no linealidades e interacciones (la ubicación fina, la cola de capacidad), y eso basta para ser los mejores con claridad.

5.3 Elección del modelo final

La regla fijada es elegir el **menor RMSE medio de CV en el escenario B**: **CatBoost** (0,4162 frente a 0,4191 de HGB). La ventaja es consistente (CatBoost gana en los 5 folds) pero pequeña: Δ = 0,0029, menos que un error estándar. Un **TOST pareado** con margen ±0,005 (fijado antes de calcular) concluye que ambos son **equivalentes en la práctica** (IC 90% de la diferencia [−0,0042; −0,0016], p = 0,012). Por eso se presentan dos finalistas: **CatBoost como modelo de referencia y el que se entrega**, y **HGB como alternativa equivalente y unas 20 veces más rápida** de entrenar.

5.4 Resultados en el test

El test se abrió una sola vez, tras comprobar su MD5, para los dos finalistas a la vez. Los intervalos de confianza se obtienen por *bootstrap* remuestreando **anfitriones** (2.000 réplicas).

Tabla 2. Métricas en el test (7.384 anuncios) frente a la CV. Fuente: `outputs/resultados_test_*.csv`.

Métrica	CatBoost test (IC 95%)	CatBoost CV	HGB test (IC 95%)	HGB CV
RMSE en log	0,429 (0,408-0,451)	0,416	0,424 (0,407-0,442)	0,419
R^2 en log	0,636 (0,601-0,672)	0,657	0,645 (0,608-0,680)	0,653
MAE ($)	49,9 (44,5-56,3)	47,6	49,4 (44,7-55,0)	48,1
Mediana del error ($)	21,1 (19,1-24,1)	21,5	21,1 (19,4-24,0)	21,5

- **Coherente con la CV:** el test sale algo peor (0,013 en CatBoost, 0,005 en HGB), dentro de la variabilidad del muestreo.
- **En el test el orden se invierte** (HGB 0,424 frente a CatBoost 0,429), pero la diferencia pareada no es concluyente: −0,0053 con IC 95% [−0,017; +0,003], que contiene el cero.
- **Error típico frente a error medio:** la mediana del error (21$) es menos de la mitad del MAE (50$): pocos anuncios caros con errores grandes elevan la media.

5.5 El rango de precio

Los cuantiles 0,1 y 0,9 sin calibrar cubrían solo el 72% en la calibración: los cuantiles extremos quedan demasiado cerca del centro en datos nuevos. La **regresión cuantílica conformal** (CQR) los ensancha lo justo.

- **Cobertura en el test:** 77,9% (CatBoost, IC 72,3-81,5) y 79,0% (HGB). Anchura mediana: 94$.
- **Por distrito:** Manhattan se queda corto (73,7%) y tiene los intervalos más anchos (125$), es la ciudad con mayor lujo es normal que su rango sea el mayor; Brooklyn y Queens cubren el 81-83%.
- **Por quintil de precio:** 84-88% en los quintiles centrales, **55,9% en el más caro**. El quintil se define con el precio real, lo que favorece mecánicamente peor cobertura en los extremos.

5.6 Dónde se equivoca el modelo

Tabla 3. Error absoluto medio de CatBoost en el test por segmento. Fuente: 4-MODELO_FINAL, §6.

Segmento	MAE ($)
Habitaciones privadas	25
Pisos enteros	70
Manhattan	66
Pisos para 7 o más huéspedes	172
Habitaciones de hotel (57 anuncios)	175
Quintil de precio más caro	154

- **Los 20 peores errores son pisos enteros de más de 1.490$ y que han infravalorados**, prácticamente los mismos en los dos modelos. El peor es un anuncio de 6.500$ que se

predice en 460$: para el modelo es un piso de 7 huéspedes y 3 dormitorios como cualquier otro, a parte de esos atributos no se detecta el lujo del piso.

- **Sensibilidad a la cola:** sin los precios por encima del percentil 99 del train (794$, 81 anuncios), el RMSE baja a 0,395 y el MAE a 39,1$. La cola explica unos 11$ del MAE.
- **Sin sesgo espacial claro** (Figura 8): los residuos grandes están repartidos por Manhattan y Brooklyn, donde hay más anuncios y precios más altos.

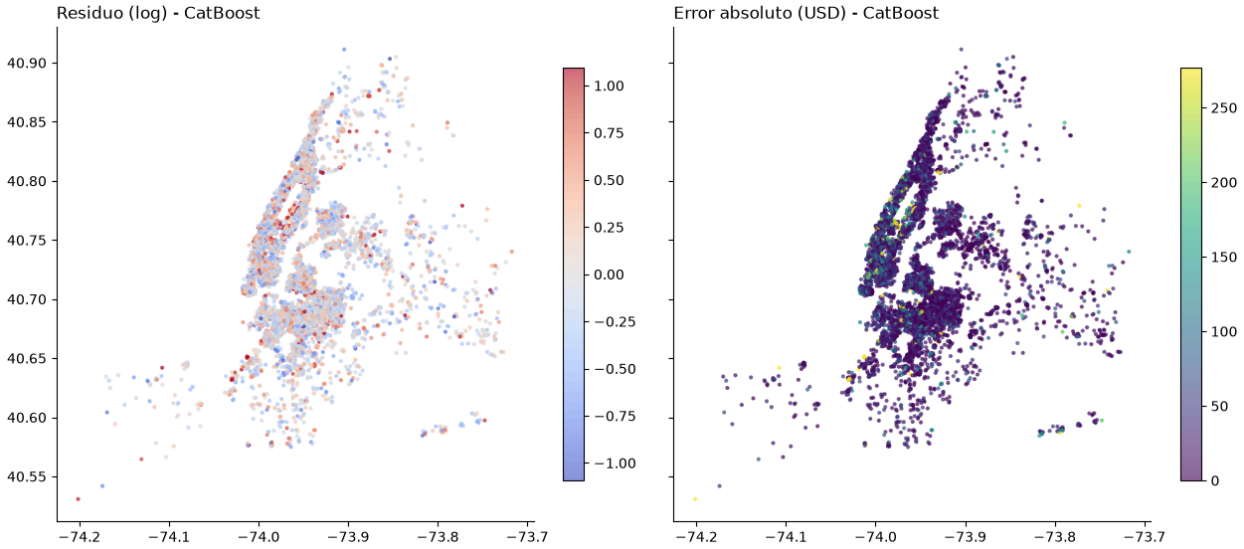


Figura 8. Los errores de CatBoost no forman manchas geográficas: la ubicación está bien capturada. Fuente: 4-MODELO_FINAL, §6.1.

6. Detalles del modelo

6.1 Variables y su poder predictivo

Las 79 candidatas se organizan en cinco bloques:

Bloque	Variables	Ejemplos
Producto	5	`room_type`, `property_type_grp`, `accommodates`, `bedrooms`, `es_estudio`
Ubicación	5	`latitude`, `longitude`, `dist_centro_km`, `district`, `neighbourhood`
Condiciones de reserva	3	`minimum_nights`, `estancia_min_30`, `instant_bookable`
Perfil del anfitrión	10	antigüedad, cartera, superhost, tasas de respuesta y aceptación, `host_ambito`, `reputacion_cartera`
Amenities comunes	56	gimnasio, ascensor, lavadora, chimenea, cocina…

Selección de variables. Se aplicó una regla fijada antes de ejecutar: una variable o bloque entra si, sobre las 15 estimaciones , Δ = RMSE sin ella − RMSE con ella es positivo en al menos 12 y el Δ medio supera su desviación típica. Es una heurística declarada como tal (los 15 Δ no son independientes); el **p de Nadeau-Bengio** se muestra como dato informativo.

La selección solo decide las variables del lineal y del kNN, a los que una variable inútil sí perjudica.

Importancia por permutación. Se mide en validación (no en train) y por bloques, porque las variables de ubicación están correlacionadas entre sí y su importancia individual se reparte. El orden es el mismo en todas las familias de modelos (Figura 9): producto ≫ ubicación > anfitrión ≈ amenities ≫ condiciones de reserva. En el test, con CatBoost: producto 0,29, ubicación 0,11, anfitrión y amenities ≈ 0,05, condiciones 0,004.

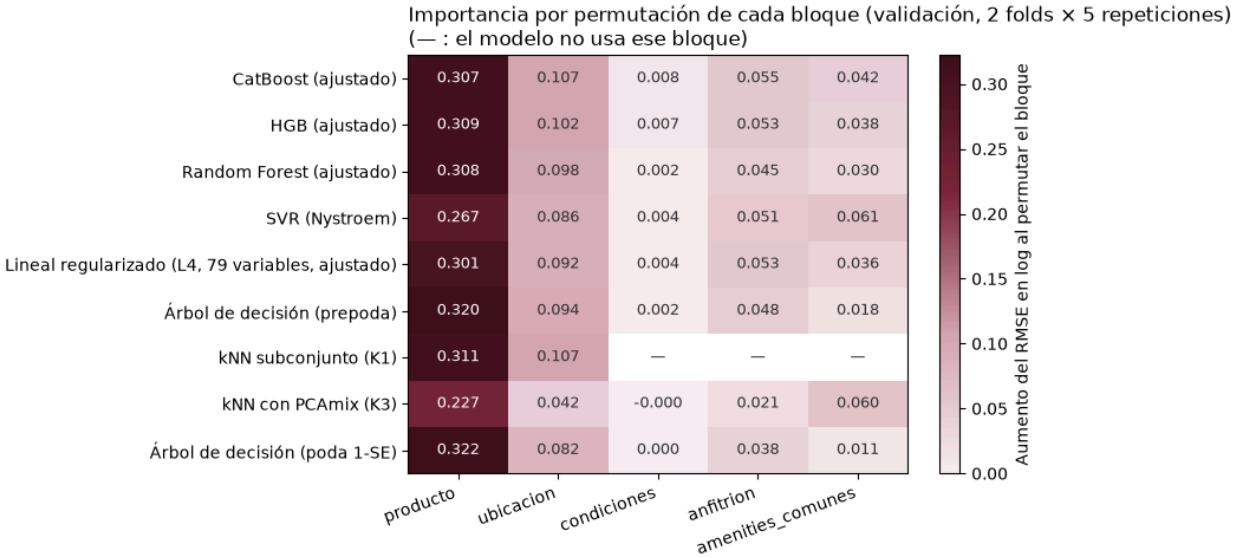


Figura 9. Todas las familias usan la misma información y en el mismo orden: el producto domina. Fuente: 3-MODELADO, §12.

Variable a variable (CatBoost, validación), encabezan `accommodates` (0,067), `room_type` (0,061), `bedrooms` (0,037), las coordenadas (0,021) y `property_type_grp` (0,020). La importancia por impureza del árbol se descartó porque premia variables continuas con muchos cortes que no generalizan (`n_anuncios_ny` era la 4.ª por impureza y la 9.ª por permutación).

SHAP. El SHAP nativo de CatBoost (Figura 10) confirma el orden: `room_type` (0,125) y `accommodates` (0,120) son las variables que más contribuyen. La contribución de la capacidad crece casi linealmente, de −0,25 con un huésped a cerca de +1,0 con dieciséis.

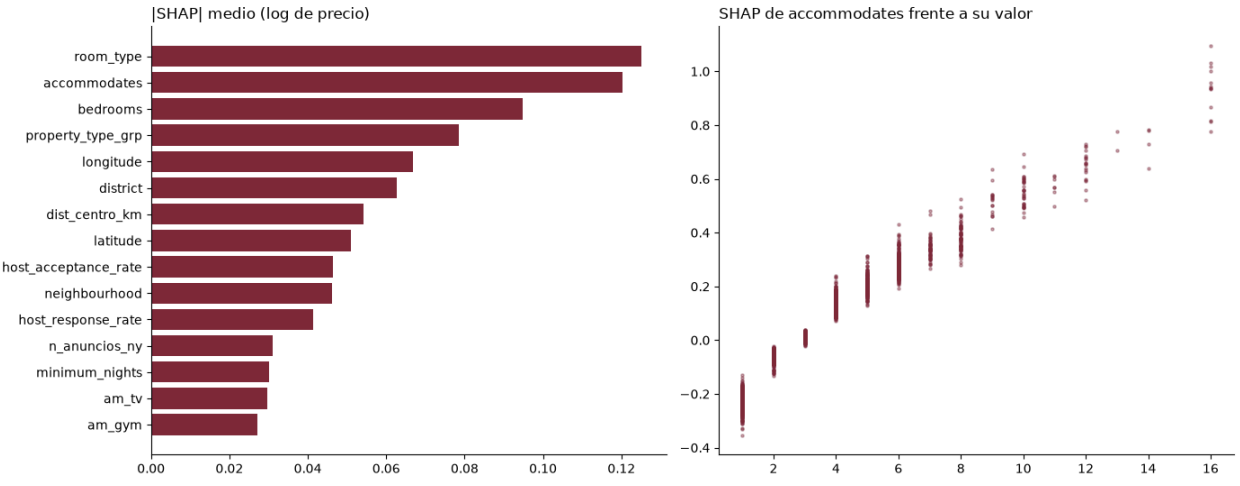


Figura 10. Tipo de alojamiento y capacidad son las variables que más contribuyen a cada predicción. Fuente: 4-MODELO_FINAL, §7.1.

Permutación y SHAP describen **cómo usa el modelo** cada variable, no un efecto causal sobre el precio.

6.2 Hiperparámetros

Búsqueda aleatoria sobre los 5 folds de la primera repetición, con cada candidato registrado fold a fold:

Modelo	Candidatos	Configuración elegida
CatBoost	10	`depth` 7, `learning_rate` 0,083, `l2_leaf_reg` 10,8, hasta 3.000 iteraciones con parada temprana a 65 sobre un 10% del train del fold; categóricas nativas con *ordered target statistics*
HGB	40	`learning_rate` 0,0225, `max_iter` 1.448, `max_leaf_nodes` 15, `min_samples_leaf` 156, `l2_regularization` 0,10, `max_features` 0,5; barrio con `TargetEncoder`

El óptimo es plano en los dos *boosting* (Figura 11): en HGB, los seis mejores candidatos están entre 0,4191 y 0,4205 con configuraciones muy distintas; lo que comparten es árboles pequeños y regularizados. En CatBoost, las ocho mejores configuraciones caben en 0,002, menos que la desviación entre folds. Por eso el RMSE de CV del ganador es algo optimista (se elige sobre los mismos folds), y por eso existe el test.

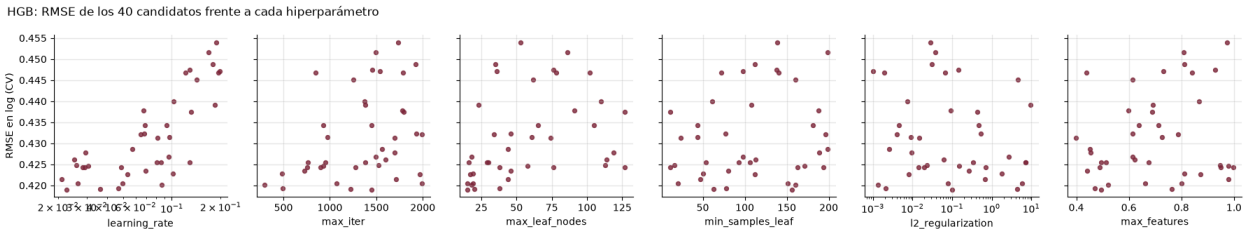


Figura 11. Solo la tasa de aprendizaje muestra una tendencia clara; el resto de hiperparámetros apenas mueve el error. Fuente: 3-MODELADO, §9.3.

Codificación del barrio. Con 220 barrios (la mitad con menos de 30 anuncios) se compararon tres opciones en HGB: `TargetEncoder` (0,4233), categórica nativa (0,4253) y sin barrio (0,4263). Ninguna diferencia pasa la regla: con coordenadas, distancia y distrito, el barrio apenas añade información. CatBoost aporta una cuarta vía, *ordered target statistics*, que evita que el precio de un anuncio entre en su propia codificación.

6.3 Eficiencia de ejecución

- **Reentrenamiento final con todo el train:** CatBoost unos **220 s**, HGB unos **10 s** (≈ 20 veces más rápido).
- **Por fold en CV:** HGB ajustado unos 7-8 s; el ganador de HGB podía sustituirse por un candidato que tarda 1,3 s con solo un 0,1% más de error. Los tiempos de CatBoost en CV se midieron con folds en paralelo y no son comparables.
- **Predicción:** inmediata en los dos).
- **Coste del plan completo:** del orden de 5 horas de cómputo tras recortar la búsqueda de CatBoost y la ablación. Los resultados se guardan en el registro y no se recalculan al reejecutar los notebooks.
- **GPU:** CatBoost en Colab acelera la búsqueda, pero no reproduce exactamente la CPU (otra discretización); el resultado oficial es el de CPU, expiro la sesion de Google Collab.

6.4 Ventajas e inconvenientes

	CatBoost (referencia)	HGB (alternativa)
Ventajas	Menor RMSE de CV, gana en los 5 folds; categóricas nativas sin fuga de la codificación; SHAP nativo	20 veces más rápido; incluido en scikit-learn (sin dependencia adicional); mismo rendimiento en test
Inconvenientes	Más lento; se ajusta más al train (0,318 frente a 0,416 de CV) sin generalizar mejor en test	Barrio con `TargetEncoder`, cuyo *cross-fitting* interno es por filas y no por anfitrión (efecto pequeño, se declara)

6.5 Limitaciones que se declaran

- **La CQR usa un solo conjunto de calibración** (fold 0) y la intercambiabilidad solo se cumple de forma aproximada (anuncios correlacionados por anfitrión).
- **La regla de precios de bloqueo se aplicó también al test;** en producción el precio no se conoce de antemano, así que el modelo no está pensado para tarifas de bloqueo.
- **Sesgo de selección por reseñas:** los pisos enteros sin reseñas son un 11% más caros; las reseñas quedaron fuera del modelo, pero cualquier análisis basado en ellas hereda ese sesgo.
- **Los RMSE de CV de los modelos ajustados son algo optimistas** (elegidos sobre los mismos folds); el test lo acota.

7. Recomendaciones

Para la organización

- **Usar el modelo como sugerencia de precio al publicar**, mostrando la mediana y el rango, no como precio automático. El valor está en dar al anfitrión nuevo una referencia mejor que "anuncios parecidos".
- **Señalar los anuncios grandes o de gama alta** (pisos para 7 o más huéspedes, predicciones altas, Manhattan) para revisión humana: ahí el rango cubre solo la mitad de las veces.
- **Si el coste de cómputo pesa, desplegar HGB**: rendimiento equivalente.

Implementación técnica

- **Servicio de predicción:** la cadena completa ya está empaquetada (anuncio en bruto → `PreparadorListings` → modelo → `exp(ŷ)` e intervalo CQR) en `predecir_precio` (`scripts/func_final.py`). Requiere pasar la cartera completa del anfitrión en Nueva York, porque `n_anuncios_ny` y `reputacion_cartera` se calculan sobre las filas recibidas; en producción conviene calcularlas desde la base de datos de anfitriones.
- **Versiones fijadas:** los modelos `joblib` solo cargan con las mismas versiones de scikit-learn y CatBoost (anotadas en `models/ficha_modelo_*.json`). Conviene servirlos en un entorno congelado.
- **Reentrenamiento periódico** con instantáneas nuevas: el modelo se ajustó con una única foto en plena pandemia. `PreparadorListings` permite preparar instantáneas nuevas con los parámetros aprendidos o reajustarse.
- **Monitorización:** cobertura del intervalo por segmento (alerta si baja del 75%), deriva de las variables de producto y ubicación, y proporción de anuncios en el segmento de lujo.

Mejoras de datos y de modelo

- **Capturar información que distinga el lujo:** superficie, fotos, texto del título y la descripción, calidad del edificio. Es la vía con más margen: el error se concentra en anuncios que las variables actuales no separan de un piso simplemente grande.
- **Tomar el perfil del anfitrión en el momento de publicar**, para que el escenario B no use información de 2021.
- **Líneas futuras:** variables del título, reseñas como complemento en un escenario de anuncios existentes, otra ventana temporal y una validación de transferencia a otra ciudad con precio en USD.

Documentación para el sponsor

Un precio de partida para cada anuncio nuevo de Airbnb en Nueva York

Resumen ejecutivo

- **Qué se ha construido:** una herramienta que, a partir de lo que un anfitrión sabe al publicar (cómo es el alojamiento, dónde está y quién lo ofrece), propone un **precio por noche** y un **rango razonable** alrededor.
- **Cuánto acierta:** en un anuncio típico, el precio sugerido se desvía **unos 21$** del precio real. Frente a fijarse en anuncios del mismo tipo y tamaño, que es lo que haría un anfitrión hoy, el error típico baja de 29$ a 21$.
- **El rango es fiable:** el precio real cae dentro del rango propuesto en **casi 8 de cada 10 anuncios**, como se buscaba.
- **Qué mueve el precio:** sobre todo el tipo de alojamiento y su tamaño; después, la ubicación exacta. El perfil del anfitrión y el equipamiento pesan menos, y las condiciones de reserva, casi nada.
- **Dónde no fiarse:** en los alojamientos de lujo. Con la información de un anuncio, un piso de lujo y uno simplemente grande se parecen demasiado, y el modelo tiende a quedarse corto.
- **Recomendación:** adoptar la herramienta como **sugerencia de precio al publicar**, con el rango visible y revisión humana en los anuncios grandes o de gama alta.

1. Objetivos del proyecto

Situación. Quien publica por primera vez un alojamiento en Airbnb tiene que decidir cuánto cobrar por noche, y lo habitual es mirar anuncios parecidos de la zona. Es una referencia imprecisa: dos pisos que parecen iguales pueden tener precios muy distintos según la calle, el edificio o el equipamiento.

Complicación. El precio depende de muchas cosas a la vez, y no siempre de la forma que parece a simple vista. Por ejemplo, a primera vista los pisos con más huéspedes por dormitorio parecen más caros, pero es solo porque son pisos más grandes: a igual capacidad, más dormitorios significa más precio. Combinar decenas de factores a mano no es posible; un modelo que aprenda de miles de anuncios sí puede hacerlo.

Objetivos.

1. Proponer un **precio por noche** para un anuncio nuevo con la información disponible al publicarlo.
2. Acompañarlo de un **rango** que contenga el precio real en 8 de cada 10 casos.
3. Explicar **qué determina el precio** y en qué tipo de anuncios se puede confiar en la propuesta.

Un cambio de alcance. El encargo partía de datos de diez ciudades. Al analizarlos se vio que el precio de cada ciudad venía en su moneda local sin indicarlo, y que el mercado funciona de forma distinta en cada una: en Bangkok una habitación privada cuesta casi lo mismo que un piso entero, y en Sídney menos de la mitad. Mezclar diez mercados en una sola herramienta habría dado peores resultados en todos. Se decidió centrar el trabajo en **Nueva York**, que tiene

el precio en dólares, la información de ubicación más detallada del conjunto y casi 37.000 anuncios.

2. Principales logros obtenidos

- **Un precio sugerido con un error típico de unos 21$ por noche**, comprobado en anuncios y anfitriones que la herramienta no había visto nunca.
- **Un 22% más preciso que la regla de "anuncios parecidos"** del mismo tipo y tamaño.
- **Un rango que cumple lo prometido** en el conjunto del mercado: el precio real cae dentro en casi 8 de cada 10 anuncios.
- **Un mapa claro de qué mueve el precio**, útil por sí mismo para orientar a los anfitriones.
- **Un diagnóstico honesto de sus límites:** se sabe dónde acierta y dónde no, lo que permite usarla con criterio.

3. Aproximación elegida

El problema se planteó como **estimar el precio de un anuncio a partir de lo que se sabe al publicarlo**. Esto obliga a una decisión importante: **no se usan las reseñas ni las puntuaciones**, porque un anuncio nuevo todavía no las tiene. Una herramienta que dependiera de ellas no serviría para el caso que interesa.

Se utilizó la información de cada anuncio agrupada en cinco tipos:

Tipo de información	Ejemplos
El alojamiento	piso entero o habitación, número de huéspedes, dormitorios, si es un estudio
La ubicación	distrito, barrio, distancia al centro, posición exacta
Las condiciones de reserva	estancia mínima, reserva inmediata
El anfitrión	antigüedad, cuántos anuncios gestiona, rapidez de respuesta
El equipamiento	56 servicios: wifi, cocina, lavadora, gimnasio, ascensor…

Para que la medida de acierto sea creíble, **una parte de los anuncios se apartó al principio y no se miró hasta el final**. Además, todos los anuncios de un mismo anfitrión se mantuvieron juntos: muchos gestionan varios pisos casi idénticos, y si uno se usara para aprender y su gemelo para comprobar, el acierto saldría inflado.

También se excluyeron los precios que no son tarifas reales, como los anuncios a 9.999$ que los anfitriones usan para bloquear fechas. Se tuvo cuidado de no eliminar el lujo verdadero, que la herramienta debe saber valorar.

4. Descripción del modelo

Qué hace	Propone el precio por noche de un anuncio nuevo y un rango en el que el precio real debería caer en 8 de cada 10 casos
Cómo funciona	Combina cientos de reglas sencillas ("si es un piso entero en Manhattan para cuatro personas…"), cada una corrigiendo los errores de las anteriores. Es un tipo de modelo muy utilizado con datos de este tipo

Con qué información	La obligatoria al subir un anuncio a airbnb
Sobre qué anuncios	Casi 37.000 anuncios de Nueva York, con datos de febrero de 2021
Cómo se comprobó	Con 7.384 anuncios de anfitriones que la herramienta no había visto nunca
Frente a qué mejora	Un 22% más precisa que fijar el precio con la mediana de anuncios del mismo tipo y tamaño

Se probaron ocho familias de modelos, desde reglas muy simples hasta modelos complejos. Los dos mejores obtuvieron resultados prácticamente idénticos; se entrega el que ganó con el criterio fijado antes de empezar, y el segundo queda como alternativa más ligera si el coste de cálculo fuera importante.

5. Resultados principales soportados por datos

Qué mueve el precio en Nueva York

Lo primero es el tipo de alojamiento y su tamaño. Un piso entero cuesta más del doble que una habitación privada (140$ frente a 60$ por noche en el anuncio típico), y cada huésped o dormitorio adicional sube el precio, aunque cada plaza extra sale más barata que la anterior.

Lo segundo es la ubicación, y no solo el distrito. El precio se concentra en el sur y el centro de Manhattan y cae hacia la periferia, también dentro de cada distrito (Figura 1). Un anuncio típico cuesta 120$ en Manhattan, 89$ en Brooklyn y en torno a 70$ en Queens y el Bronx.

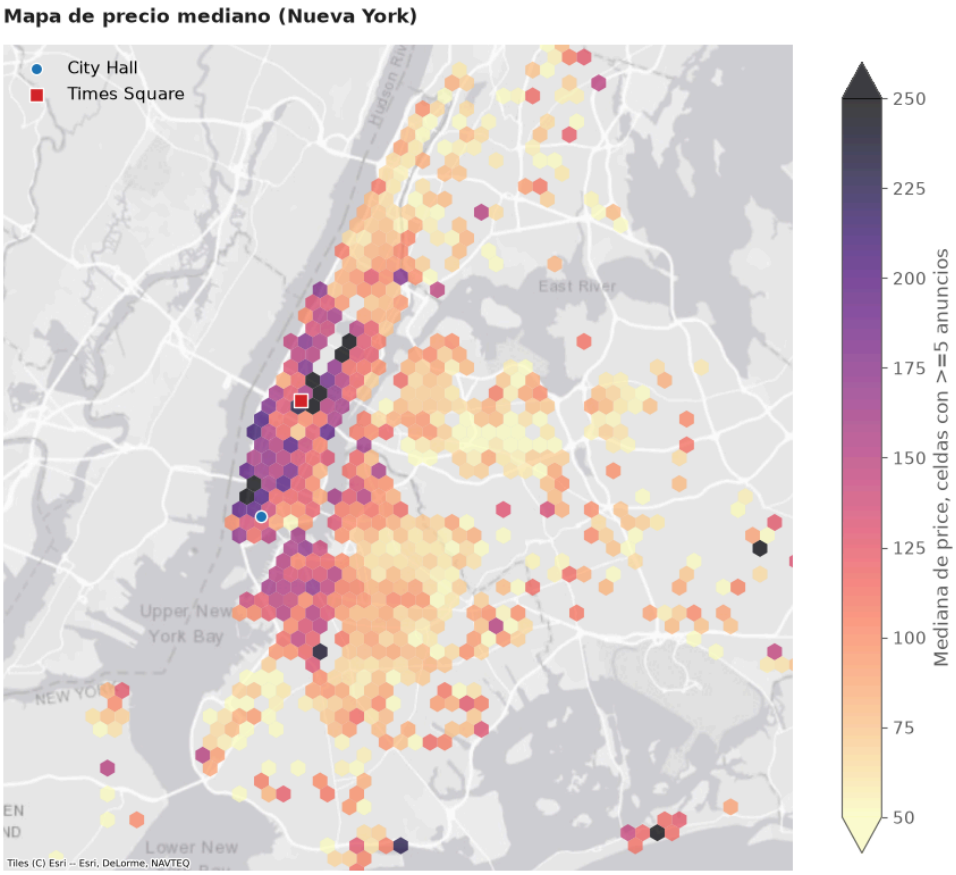


Figura 1. El precio por noche se concentra en el sur y el centro de Manhattan y baja hacia la periferia.

El anfitrión y el equipamiento aportan algo parecido entre sí, y menos. Conocer el perfil del anfitrión mejora la propuesta, pero la herramienta sigue siendo útil para un anfitrión que publica por primera vez: su error solo aumenta alrededor de un 4%. En equipamiento, lo que más se asocia a precios altos son servicios propios de edificios y viviendas de gama alta, como gimnasio o chimenea. **Las condiciones de reserva**, como la estancia mínima, apenas influyen.

La Figura 2 resume el peso de cada tipo de información: si se le oculta a la herramienta el tipo y el tamaño del alojamiento, su error sube un 73%; la ubicación, un 25%; el anfitrión y el equipamiento, entre un 10% y un 13%; las condiciones de reserva, un 2%.

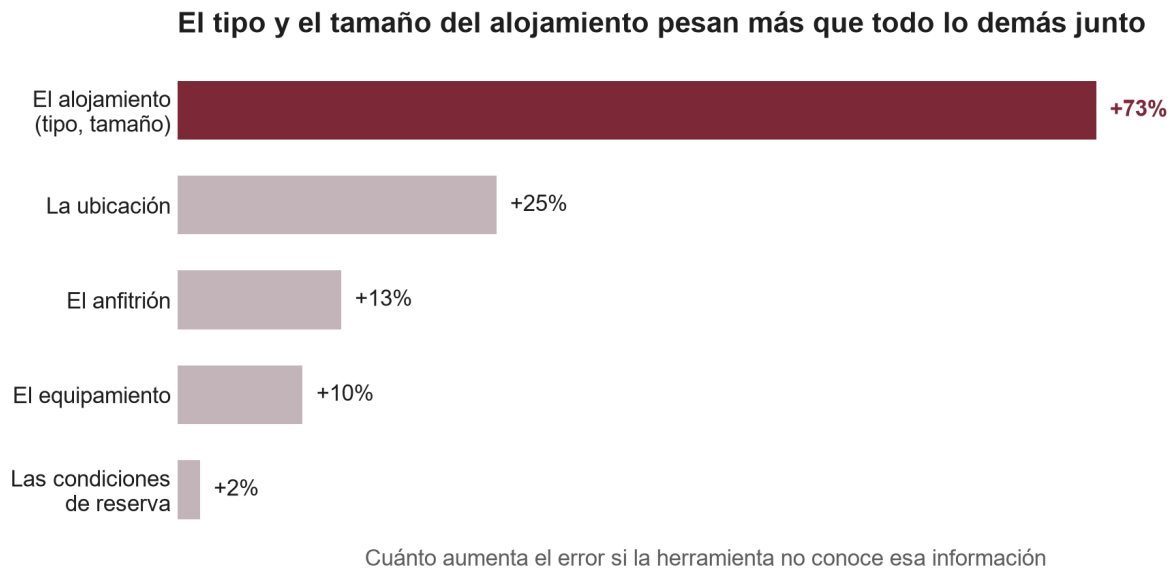


Figura 2. El tipo y el tamaño del alojamiento pesan más que todo lo demás junto.

Estas relaciones describen **cómo razona la herramienta**, no causas: que los pisos con gimnasio sean más caros no significa que instalar un gimnasio suba el precio.

Cuánto acierta

En un anuncio típico, el precio sugerido se desvía **unos 21$** del real. El error medio es mayor, unos 50$, porque unos pocos anuncios muy caros acumulan errores grandes (Figura 3).

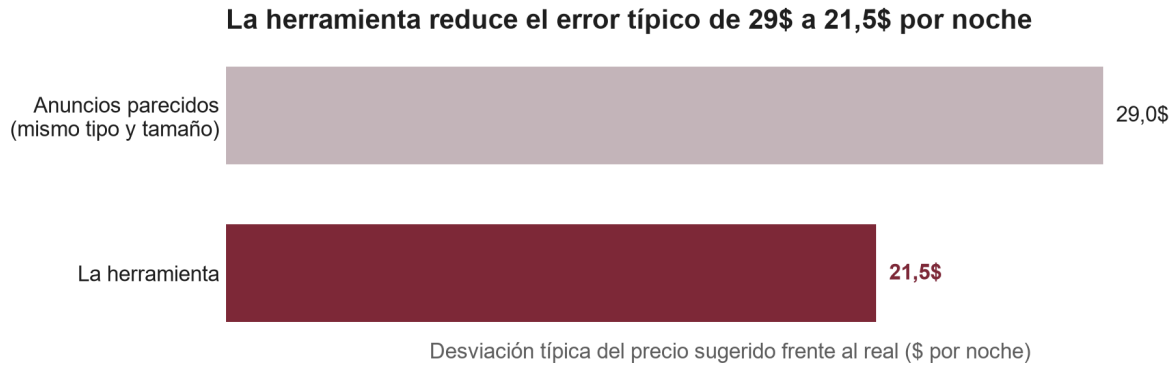


Figura 3. Frente a fijarse en anuncios del mismo tipo y tamaño, la herramienta reduce el error típico de 29$ a 21,5$ por noche (validación sobre los datos de entrenamiento).

Dónde es fiable y dónde no

En la mayor parte del mercado, la herramienta es fiable. En las habitaciones privadas el error medio es de unos 25$, y en los anuncios de precio intermedio el rango acierta en más de 8 de cada 10 casos.

En el lujo, no. En el 20% de anuncios más caros, el rango acierta solo en algo más de la mitad de los casos, y los mayores errores son siempre pisos de lujo a los que la herramienta pone un precio demasiado bajo. El caso más extremo es un piso de 6.500$ por noche al que propone 460$: para la herramienta es un piso para siete personas como cualquier otro, porque nada en el anuncio indica que sea excepcional (Figura 4).

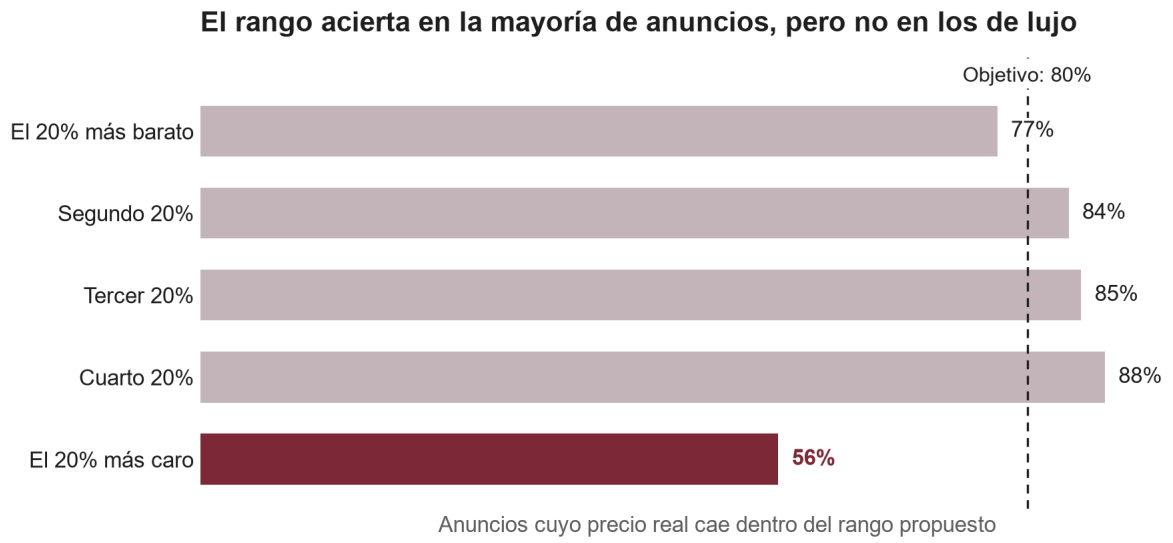


Figura 4. El rango acierta en la mayoría de anuncios, pero no en los de lujo: en el 20% más caro, solo en el 56% de los casos.

6. Detalles del modelo

El detalle técnico (cómo se eligió el modelo, qué otros se probaron, cómo se ajustó y cuánto tarda) está en la Documentación para analistas, sección 6.

7. Recomendaciones

Cómo usarla. Se recomienda incorporar la herramienta en el momento de publicar un anuncio, como **sugerencia** de precio con su rango visible, no como precio fijado automáticamente. El anfitrión conserva la decisión; la herramienta le da un punto de partida mejor que comparar a ojo.

Beneficios esperados (en nuestra opinión).

- **Anfitriones nuevos mejor orientados**, con menos anuncios que salen demasiado caros (y no se reservan) o demasiado baratos (y pierden ingresos).
- **Un mercado más coherente**, con precios más alineados con lo que realmente ofrece cada alojamiento.
- **Información útil para el equipo de negocio:** qué factores pesan en el precio de cada zona y tipo de alojamiento.

Riesgos y cómo gestionarlos.

- **Anuncios de lujo:** marcar para revisión humana los alojamientos grandes, de gama alta o con un precio sugerido alto, donde la herramienta tiende a quedarse corta.
- **Datos de un solo momento:** la herramienta aprendió de una foto del mercado de febrero de 2021, en plena pandemia. Debe **actualizarse periódicamente** con datos recientes antes y durante su uso.
- **Lectura de los factores:** lo que la herramienta asocia a precios altos no son recetas. No se debe comunicar a los anfitriones que añadir un servicio subirá su precio.

Siguiente paso con más impacto. Recoger información que distinga la calidad del alojamiento (superficie, fotos, descripción) mejoraría la herramienta precisamente donde hoy falla.

Apéndices

> Dirigidos a los ingenieros y analistas que quieran reproducir, revisar o desplegar el trabajo. El repositorio completo acompaña a esta memoria.

A. Mapa del repositorio

Repositorio del proyecto: https://github.com/hugomontilla/TFM

```
data/        listings.csv y reviews.csv (no versionados); particion/ (train y test fijos con MD5);
             modelado/ (parquet preparados y parámetros del preparador)
scripts/     código reutilizable (limpieza, variables, partición, modelado, modelo final, gráficos)
notebooks/   análisis y modelado, en orden numérico (0 a 4)
models/      modelos finalistas, cuantiles del intervalo y fichas
outputs/     registro de experimentos, folds de CV, importancias y resultados en test
docs/        memoria, bitácora de decisiones, hallazgos, registro de experimentos, planes y enunciado
.env         parámetros comunes (semilla, folds, cobertura del intervalo, réplicas del bootstrap)
```

B. Notebooks (orden de ejecución)

Notebook	Entrada	Qué hace	Salida
`0-INTRODUCCION`	`listings.csv`	Compara las 10 ciudades y justifica acotar a Nueva York	Decisión de alcance
`1-EXPLORACION`	`listings.csv` (NY)	Calidad de datos sin mirar el precio: duplicados, nulos, filas no válidas, ubicación, anfitriones. Diseña y crea la partición	`data/particion/`
`2-ANALISIS`	train	Análisis exploratorio del precio, solo con el train	Variables candidatas y reglas de limpieza
`3-MODELADO`	train	Referencias, lineal, selección de variables, kNN, árbol, ensembles, SVR, CatBoost, comparación e importancia	`outputs/experimentos.csv`, importancias
`4-MODELO_FINAL`	train + test (una vez)	Reentrena los dos finalistas, calibra el intervalo, evalúa en test, analiza errores, interpreta, guarda y compara	`models/`, `outputs/resultados_test_*.csv`

Los notebooks 3 y 4 se pueden ejecutar sin los CSV originales: la partición y los datasets de modelado están versionados. Los experimentos ya registrados no se recalculan (el registro hace de caché).

C. Módulos de `scripts/`

Módulo	Responsabilidad
`config.py`	Lee los parámetros comunes de `.env`

Módulo	Responsabilidad
`particion_datos.py`	Crea y carga la partición fija (agrupada por `host_id`, estratificada por `room_type` × `district`); escribe `particion.json` con las huellas MD5; no sobrescribe sin `--forzar`
`estudio_estratificacion.py`	Simula 50 particiones para justificar el estrato
`preparacion_datos.py`	`PreparadorListings` (`fit`/`transform`): limpieza, regla de precios de bloqueo y construcción de las 79 variables. Es la única fuente de verdad de las variables. Se ejecuta sin argumentos para regenerar `data/modelado/`
`func_aux.py`	Estadística del EDA (Welch-ANOVA, η², V de Cramér, lift por estrato, contrastes por grupo)
`func_modelado.py`	Núcleo del modelado: folds guardados, `evaluar`, preprocesadores por familia, `RegresorCatBoost`, `MedianaPorGrupo`, PCAmix/FAMD, búsqueda de hiperparámetros e importancia por permutación
`registro.py`	Único acceso a `outputs/experimentos.csv` (escritura atómica, sustituye un experimento si se reejecuta)
`func_final.py`	`ModeloFinal` (configuración desde el registro, ajuste, CQR, test con *bootstrap* por anfitrión, errores, importancia, guardado), `predecir_precio` y la diferencia pareada entre modelos
`estilo_graficos.py`	Estilo único de gráficos: un color fijo por categoría en toda la memoria
`extraer_figuras.py`	Extrae a PNG las figuras ya renderizadas de los notebooks, sin volver a ejecutarlos

D. Reproducción

```
python -m venv .venv
.venv\Scripts\activate          # Windows
pip install -r requirements.txt
python scripts/preparacion_datos.py   # solo si se parte de los CSV originales
```

- **Versiones usadas:** Python 3.13, scikit-learn 1.9, pandas 3.0, CatBoost 1.2 (registradas en `models/ficha_modelo_*.json`). Los modelos `joblib` solo cargan con la misma versión de scikit-learn y CatBoost.
- **Datos no versionados:** `data/listings.csv` (160 MB) y `data/reviews.csv` (250 MB), del conjunto público de Airbnb 2008-2021 (licencia CC0).
- **Aleatoriedad controlada:** `SEMILLA=42` en `.env`. Cambiar la semilla o el número de folds invalida el registro de experimentos y los folds guardados.
- **Integridad de la partición:** `particion.json` guarda el MD5 de `listings_train.csv` y `listings_test.csv`; el notebook final comprueba el del test antes de cargarlo.
- **GPU:** `notebooks/Entrenamiento Google Colab/` contiene una copia mínima para repetir la búsqueda de CatBoost en GPU; el resultado oficial es el de CPU.

E. Uso del modelo guardado

Para cada finalista hay tres ficheros en `models/`: el modelo (`modelo_final_<modelo>.joblib`), los cuantiles con su corrección CQR (`cuantiles_<modelo>.joblib`) y la ficha

(`ficha_modelo_<modelo>.json`, con hiperparámetros, las 79 variables, métricas de CV y test, MD5 de la partición y versiones).

```
# Desde la raíz del repositorio, con scripts/ en el sys.path (como hacen los notebooks)
import joblib, json
from preparacion_datos import PreparadorListings, RUTA_PREPARADOR, cargar_listings_ny
from func_final import predecir_precio

preparador = PreparadorListings.cargar(RUTA_PREPARADOR)
modelo = joblib.load('models/modelo_final_catboost.joblib')
intervalo = joblib.load('models/cuantiles_catboost.joblib')
columnas = json.load(open('models/ficha_modelo_catboost.json', encoding='utf-8'))['variables']

anuncios = cargar_listings_ny('data/listings.csv')       # filas en bruto con el formato de listings.csv
precios = predecir_precio(anuncios, preparador, modelo,
                          intervalo['cuantiles'], intervalo['correccion_cqr'], columnas)
# -> listing_id, precio_mediano, precio_bajo, precio_alto (USD por noche)
```

Atención: `n_anuncios_ny` y `reputacion_cartera` se calculan sobre las filas que se pasan. Para un anfitrión con varios anuncios hay que pasar su cartera completa en Nueva York, no solo el anuncio nuevo.

F. Documentación de trazabilidad

Documento	Qué contiene
`docs/bitacora_decisiones.md`	Cada decisión vigente, con fecha, alternativas descartadas y motivo
`docs/hallazgos_y_pendientes.md`	Hallazgos de calidad de datos y riesgos metodológicos (identificadores H-xxx citados en los notebooks)
`docs/registro_experimentos.md`	Versión legible de `outputs/experimentos.csv`, por familia de modelos
`docs/arquitectura.md`	Cómo está construido el proyecto e invariantes de diseño
`docs/Planes/`	Planes de modelado y de modelo final con los que se trabajó

G. Tablas complementarias

G.1 Ablación por bloques y variables

Diferencia de RMSE en log al quitar cada bloque o variable frente al modelo completo (media de 15 estimaciones), en milésimas. Positivo = el modelo empeora al quitarlo. Fuente: `docs/registro_experimentos.md`, §7.

Qué se quita	Ridge	HGB
Bloque producto	+111,2	+108,7
Bloque ubicación	+31,7	+34,7
Bloque anfitrión	+17,2	+15,1
Bloque amenities comunes	+13,9	+12,4
Bloque condiciones de reserva	+1,0	+2,5
Tasas de respuesta y aceptación	+4,7	+7,3
Latitud y longitud	+2,8	+5,2

Qué se quita	Ridge	HGB
Tipo de propiedad	+3,1	+2,0
Indicador de estudio	+2,2	+2,3
Antigüedad del anfitrión	+0,1	+1,7
Reputación de cartera	+0,4	+1,2
Ámbito del anfitrión	+0,4	+1,1
Reserva instantánea	+0,0	+0,4
Superhost	+0,2	−0,3
Identidad verificada	0,0	0,0

G.2 Importancia por permutación de cada bloque en todos los modelos (validación)

Aumento del RMSE en log al permutar el bloque. Fuente: `outputs/importancia_bloques.csv`.

Modelo	Producto	Ubicación	Condiciones	Anfitrión	Amenities
CatBoost	0,307	0,107	0,008	0,055	0,042
HGB	0,309	0,102	0,007	0,053	0,038
Random Forest	0,308	0,098	0,002	0,045	0,030
SVR	0,267	0,086	0,004	0,051	0,061
Lineal (Lasso)	0,301	0,092	0,004	0,053	0,036
Árbol (prepoda)	0,320	0,094	0,002	0,048	0,018
kNN (K1)	0,311	0,107	—	—	—
kNN con PCAmix	0,227	0,042	0,000	0,021	0,060